The `defined()` function will check if a constant already exists and return `true` if it does (and of course `false` if it doesn't). Two things to note here. The `const` keyword cannot be used unless it is in "top-level scope," which means it can't be used inside conditional blocks, functions, loops, or try/catch blocks (see Chapter 4, "Logic and Control," and Chapter 3, "Functions," for more info on what these are). In addition, `defined()` must be given the constant name as a string; otherwise, an error will be emitted if the constant being tested for doesn't exist:

```
var_dump(defined(I_AM_NOT_DEFINED));
```

```
Output:
Fatal error: Uncaught Error: Undefined constant "I_AM_NOT_DEFINED"
```

Predefined Constants

There are many predefined constants available from the moment your code starts to run, often of great use in obtaining information about the environment within which your code is executing. Here are a few examples:

```
var_dump(PHP_VERSION);
var_dump(PHP_SAPI);
var_dump(PHP_EOL);
var_dump(DIRECTORY_SEPARATOR);
```

```
Output:
string(5) "8.2.8"
string(3) "cli"
string(1) "
"
string(1) "/"
```

With so many constants already present globally, there's a chance you might accidentally attempt to redeclare one. The good news is that there are better ways to make constant data available throughout your application without polluting global data; take a look at Chapter 6, "Object Fundamentals," for ways in which classes and the new `enum` type can provide constants.